From a Lean source to a reviewable explanation

An architectural companion to Lean Exposition

25 October 2026 · revised draft, not independently reviewed

Draft scope. This companion describes the local file and query contracts. Its worked trace demonstrates recorded correspondence, not mathematical certification or an observed improvement in reader understanding.

1. What a reader needs to recover

A reader reaches the sentence "the gap cannot occur." They need to know which hypotheses apply, where the strict inequality is proved, what the formal statement covers, and which edition they are reviewing. A Lean proof can still compile after an author changes the answer to any of these questions in prose. Source links help; the remaining task is to compare the particular claim, argument and checking record rather than infer support from a shared title.

Lean Exposition makes the author's proposed correspondence inspectable. One local graph records claims, typed relations, source editions and checks. Three authored mathematical accounts have separate passage maps into that graph. The command-line and Python consumers expose those records together with the exact text. Authors and reviewers still have to judge whether the proposed correspondence is right. The tools neither write the argument nor discharge its mathematical obligations.

The example is full support in the zero-temperature Sherrington–Kirkpatrick (SK) model, which maximises random pair interactions over signs. Its Parisi formula expresses the limiting optimum as a minimum over nonnegative, nondecreasing, right-continuous, integrable functions $\gamma : [0,1) \to [0,\infty)$. Their Stieltjes measures satisfy $\mu_\gamma([0,t]) = \gamma(t)$; an atom at zero is allowed and no probability normalisation is imposed. The statement to follow is:

For the pure SK model at zero external field, every admissible minimiser has $\operatorname{supp}_{[0,1)} \mu_\gamma = [0,1)$**.** Every nonempty relatively open interval therefore receives positive mass. Minimiser existence is an imported theorem, separate from this conditional conclusion.

The explanation crosses a formal statement, ordinary analytic estimates and further consequences. OpenAI supplies the two source manuscripts and selected Lean development; the mathematical results and antecedents retain their literature credit. Our work here is the reconstructed accounts, their passage correspondence and the local consumer. Positive density, a scalar value formula and circuit convergence for the quantum approximate optimization algorithm (QAOA) each need further arguments; they cannot inherit verification from the words "full support."

2. Follow the gap argument

For an admissible minimiser, let $q(t) = \mathbb{E}[\Phi_x(t, X_t)^2]$, the second moment of the value function's spatial gradient along its diffusion. Suppose a bounded support gap (ℓ, r) has positive constant coefficient. Contact gives $q(r) - q(\ell) = r - \ell$ and $q'(\ell), q'(r) \leq 1$. A separate analytic estimate gives $q''' > 0$. Thus q' is strictly convex, lies below one inside the gap, and has integral smaller than $r - \ell$: a contradiction. Zero coefficients and initial or terminal gaps require separate arguments.

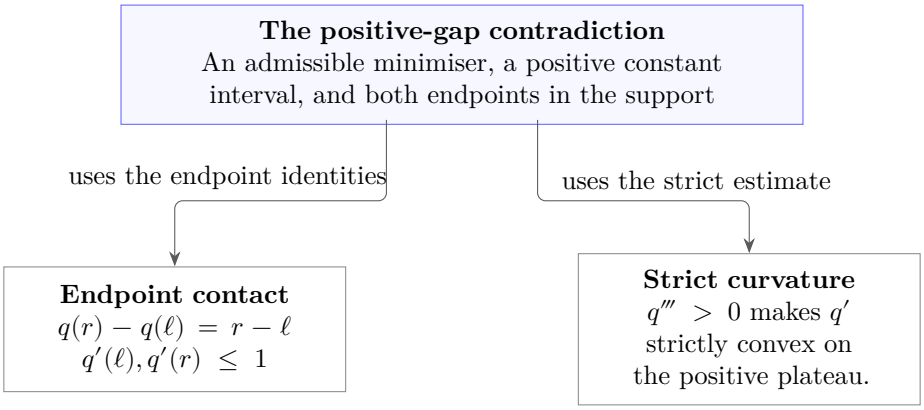


Figure 1: A selected argument view. Arrows point from the contradiction to the ingredients it uses. This is the positive-coefficient bounded-gap branch only; zero coefficients and initial and terminal gaps have separate arguments. The figure adapts the shared-support composition in the Plectis Erdős systems paper. Its relations are curated from the original proof, not extracted Lean dependencies.

The consumer guide gives the assembly command for the supplied papers. With the result in `build/qaoa-sk-authored`, ask which short-paper passage uses curvature:

```
lean-exposition query-papers build/qaoa-sk-authored \
  --paper short --passage main-result \
  --node support.positive_curvature --relation prerequisites
```

A local run against the revised authored bundle returned the following values (selected fields; JSON nesting omitted):

```
anchor: main-result
evidence_class: source_argument
root_node_id: support.full_relative_support
target_node_id: support.positive_curvature
review_state: needs_revision
semantic_certification: false
```

Its passage correspondence and returned witness links give this exact route:

```
short / main-result
  --maps_claim--> support.full_relative_support
  --supports--> support.no_bounded_gap
  --supports--> support.positive_curvature
```

The view retains `assumptions.full_support_minimizer`, including admissibility and minimality on a Brownian system. The curvature node is `unverified`; its associated source-byte check is `passed`, while the recorded semantic review is `not_run`. Its locator is `curvature.tex:9-52; prop:gap-curvature`. These are distinct returned records, not a single theorem-verification badge. The captured manifest SHA-256 is:

`9e18e224a20e4a57663497a44735399058c38c1d03adf33d20beff1b1d3514e9`

A successor assembly has a new identity; keep the requested digest with the review and use `--expect-manifest` to reject a different edition. The query reads embedded records, not the external source or receipt files.

Here `maps_claim` is an assertion in a passage map; `supports` is an authored claim-to-ingredient relation. The curvature source locator leads to OpenAI's `prop:gap-curvature`. The long record's "Strict curvature on a constant interval" reconstructs the signs, integrated coercivity and approximation behind that estimate. Read it to recover the strict inequality under its plateau hypotheses, then return to the short paper's gap argument: contact supplies the endpoint data used with that inequality. "Excluding terminal, interior and initial gaps" supplies the other cases. Finding the curvature passage does not explain its proof, and a rational positivity certificate alone does not justify the analytic estimate.

The formal route is separate: `full_support` is recorded as `formalizes` → `lean.full_support`. The selected challenge and implementation assert measure and diffusion existence, full relative support, strict increase and two moment identities, conditional on a supplied admissible minimiser. They do not assert minimiser existence, positive density or circuit convergence. No formalisation edge from the curvature claim is invented by this walkthrough.

Correspondence can itself require an argument. The long record's formal-interface discussion distinguishes a formal limit along a prescribed sequence of step sizes from the manuscript's ordinary right derivative, which covers all positive step sizes tending to zero. It supplies the intervening convexity argument. That mathematical bridge must be read and assessed; following a `formalizes` edge cannot perform it.

Omitting `--relation prerequisites` instead asks whether curvature is **directly mapped** in this passage. The theorem passage has no such direct match; its recorded support reaches curvature. A passage can use an ingredient without explaining it. Prerequisite queries follow recorded Lean references, curated support, formalisation correspondence and explicit assumptions, retaining each kind in the witness path. Citations, platform decomposition and document order do not enter that traversal. Reachability does not certify that the ingredients suffice.

The long record's **"An exact one-step calculation"** is a useful negative test. It uses $\gamma = 0$, which is admissible but not a minimiser. Its map selects conditional rounding, not the minimiser-dependent asymptotic construction. The calculation and a valid hash cannot supply the missing minimality premise. This is a judgment about the map's meaning, not a defect traversal can repair.

3. Write three accounts from the same sources

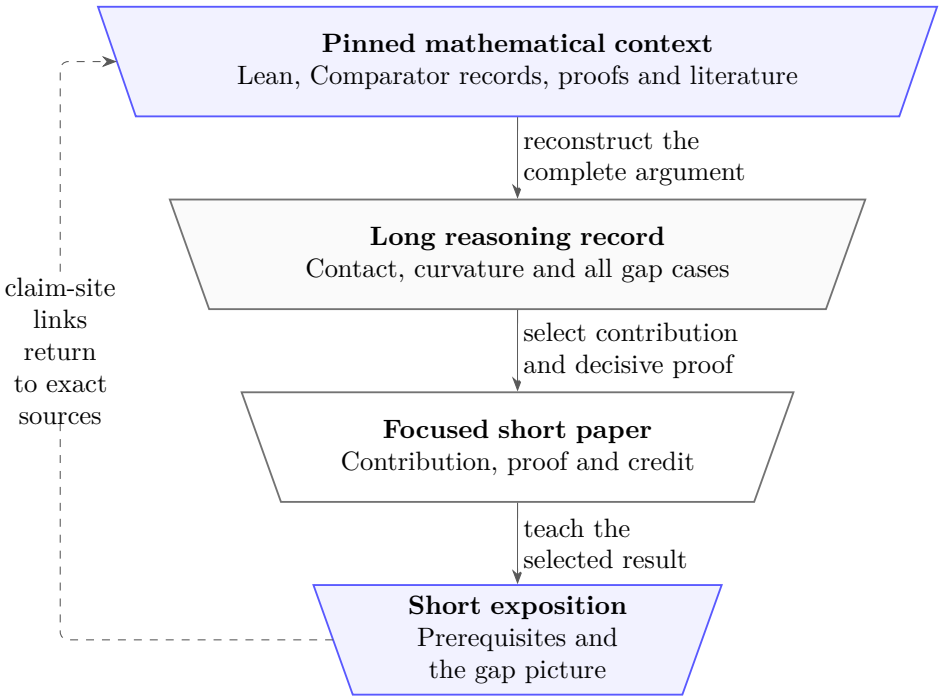


Figure 2: Three writing tasks, adapted from Will Cook's existing mathematics-map visual. Widths indicate editorial selection, not page counts or proof strength. Solid arrows name authored transitions; the dashed arrow follows source correspondence. Each writer retains all earlier inputs. Neither route certifies mathematical meaning.

The three writing skills guide people and agents; they are not runtime stages. The long writer reads the selected Lean definitions and proofs, original arguments, literature and Lean/Comparator records to reconstruct a complete selected argument. The expert-paper writer retains all those inputs and the long account, then selects the contribution and decisive proof. The exposition writer also reads the short paper and chooses one meaningful result to teach at stated prerequisites. Inputs accumulate; neither later paper has to cover every earlier result.

Entry	What that account adds
Long record, "Strict curvature on a constant interval", passage `curvature`	The analytic derivation, boundary control and passage to an integrable coefficient
Short paper's gap proof, passage `gap-proof`	The decisive contradiction and distinct gap cases
Exposition, "Why a flat interval cannot minimise", passage `contact-and-gap`	The contact and area argument at its declared reader prerequisites

Their maps retain the setting and assumptions of each selected claim. Curvature itself does not require minimality; contact and the no-gap conclusion do. Matching the same graph edition must not flatten those differences. Authors follow omitted proofs to their exact conclusion, then return to the shorter account. Each draft is also read in sequence: a later definition cannot repair an earlier unexplained step for the first-time reader.

A graph can index the long record's existing passages. It cannot determine a sufficient proof order or supply missing prose. Markdown is the prose authority; maps identify its exact passages. The paper view's `document_parts` partitions all source bytes into mapped passage blocks and surrounding unmapped text. `reconstruct_paper` requires the expected manifest and those unchanged parts, then returns the original Markdown bytes, preserving markers, definitions, headings and references. It rejects changed, omitted or reordered parts. This recovers an authored record; a rewritten argument needs fresh editorial judgment.

4. What the backend returns

The implemented backend offers three scales of inspection from the same data:

Reader task	Existing Python entry and CLI
Inspect every captured graph record and all three papers	`project_corpus` / `inspect-corpus`
Follow one kind of relation from selected nodes	`project_graph` / `inspect --view`
Open a paper or passage and its recorded support	`project_paper` / `inspect-bundle --paper`; `query_papers` / `query-papers` also selects sections and finds cross-paper occurrences

The API contract specifies signatures and versions. The corpus projection retains unselected and disconnected records in one shared pool and separates paper focus, support membership and direct passage correspondence. A local typed view selects declaration, argument, citation, platform or provenance relations. Reverse consumer traversal preserves each recorded edge's direction and meaning.

A paper subgraph retains selected claims and assumptions, recorded prerequisites and associated source/evidence records. Document links separately record heading structure, passage overlap and reading order. Whole passages remain available when a section cuts across them; unmapped text stays visible. Locations are half-open **UTF-8 Markdown byte spans**, with line positions. They are not PDF highlight boxes, browser character offsets or a rendered-page map. A rendered edition needs its own identity and tested source mapping.

A graph's `supports` arrow runs from claim to ingredient; `formalizes` runs from informal claim to formal declaration; `cites` identifies a reference. Lean type, value and structural references, platform decomposition and field-derived provenance retain their separate kinds. Strongly connected components summarise reachability in the chosen relation set: a reference cycle may be harmless, while circular proof justification still needs examination. "Whole" means the whole captured graph, not all Lean declarations or a complete mathematical dependency audit.

These local read-only projections require no second graph or prose database. A visual reader is not part of the delivered implementation; the API results themselves are the demonstrated inspection surface.

5. Checks, revisions and review

A source hash identifies bytes. Extraction records references visible in a particular Lean environment, with hidden-body and scope limits. A build checks specified source in that environment;

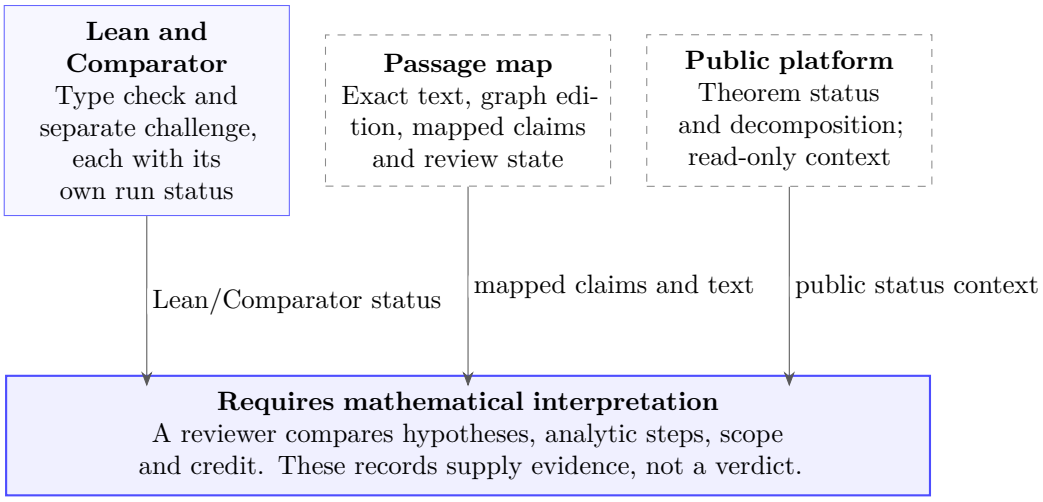


Figure 3: Three distinct records can inform a later human review. Their arrows specify what each channel supplies; none certifies mathematical interpretation or says review has occurred. Adapted from the two-correspondence figure in the Plectis Erdős systems paper.

Comparator adds a separately stated challenge and axiom policy. Public platform metadata records another service's observation. None establishes that English matches the sources or that a reader understands it.

For the QAOA/SK evidence reconciled on 25 October 2026, source-byte checks passed, but the corrected replay was interrupted without a recovered terminal verdict. Its Comparator and research extraction outcomes remain unknown. Historical attempts exported the challenges and then failed during solution builds because compiled imports were missing. This snapshot cannot be cited as successful Comparator acceptance; it also supplies no mathematical counterexample. A new configuration or later replay needs its own source-bound result. The verification guide retains the details; Lean's versioned validation guide explains general trust boundaries. Its later interface does not establish a run for the source's Lean 4.34.1 environment.

Suppose a paragraph stays unchanged while its map is revised from an unverified analytic argument to a selected formal statement. The words are identical, but the claimed correspondence has changed. Its mapping and manifest hashes change; a review of the earlier edition does not accept the new one. A changed source or hypothesis likewise needs reassessment, even when a claim's name is retained.

A review account identifies reviewer role, exact edition and claim, question, sources actually inspected, decision and limits. Preserve the original response, the author's disposition and any later confirmation separately. The current drafts have no independent human acceptance. A proposed trial asks a newcomer to recover the gap's hypotheses, analytic source, formal scope and unresolved review status from one frozen edition. Separately ask them to explain how the endpoint data and strict curvature contradict the required integral. Locating an ingredient and recovering its use are different outcomes. That trial has not been performed here.

6. Why use separate texts and maps?

The exchange format has eight files: a manifest, a graph and three Markdown/mapping pairs. The manifest binds the other seven files; each mapping binds its paper and passages to the graph edition and recorded review state. Separate maps keep ordinary Markdown readable while correspondence metadata can be revised explicitly. Eight is an implementation choice, not a lower bound: embedded annotations or one container with separately addressable accounts could preserve many of the same invariants.

Alternative or antecedent	Relationship to this design
Paper with source links	Supplies readable prose and citations. The extra contract here exposes typed recorded paths and mapping-only revisions to local queries. No comparative efficiency result is claimed.
leanblueprint	Authors already link informal statements to Lean declarations and specify dependencies. This bundle selects a three-account passage contract; it does not originate prose-to-formal links.
TheoremGraph v1, "Formal Graph," "Bridging The Corpora" and "Limitations"	Extracts typed declaration dependencies and proposes statement matches using embeddings and an LLM judge. Its match category includes specialisations and one-way implications. Here passage maps are authored assertions; neither mechanism proves semantic equivalence.
Prove2Me v2, §§3–5	Chen, Marwaha, Lu, Yuen and Peng distinguish immutable statements, proof submissions, explanations, decomposition and human audit. Its case studies do not evaluate this local bundle.

The pinned source study also records LeanArchitect's collector, doc-gen4's index and Verso Blueprint's source/presentation boundaries. The extractor adapts Comparator structural traversal with its Apache-2.0 notice. The scholarly foundations are de Moura and Ullrich's Lean 4 paper and The mathlib Community. These are specific design and software credits, not claims that related systems lack an unassessed feature.

The cost is manual correspondence work and review across three texts. Portability means offline inspection of embedded text, graph and records. External Lean sources, manuscripts and receipt files are not included; locators and hashes do not guarantee later retrieval. If those bytes are unavailable, that source check remains unavailable. The optional Prove2Me adapter reads public metadata; it does not submit a paper or obtain its review.

Figure sources. The funnel adapts Will Cook's mathematics-map visual. The support and evidence figures reuse the shared-support and two-correspondence compositions in his *A Repository-Based System for Research and Publication* (CC BY 4.0). The authoring guide locates the public writing methods adapted for the three transitions.